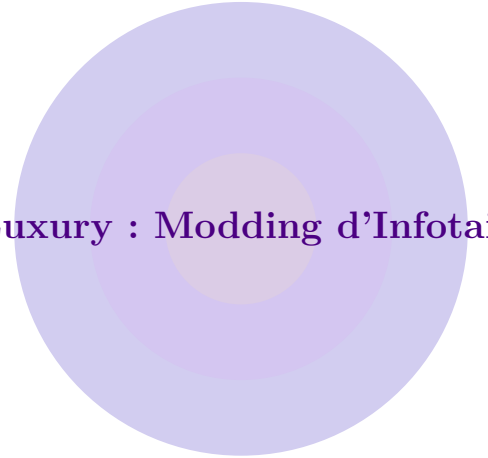
Projet personnel
Launcher Android Custom

Documentation complète

Accès au système, développement Android, ADB over WiFi
Architecture Java et dépannage alarme vitesse



MG4 Luxury : Modding d'Infotainment

Par : a1ka

Launcher Android 9 (API 28) — SAIC MG4

9 mars 2026

Table des matières

1 Comprendre le système de la MG4 **4**
1.1 Analyse de l'OS . . . 4
1.2 Pourquoi peut-on installer un APK dessus ? . . . 4
1.3 Ce qu'on ne peut PAS faire sans root . . . 4

2 Prérequis développement **5**
2.1 Logiciels & Interfaces nécessaires . . . 5
2.2 Compatibilité Java / Gradle . . . 5

3 Accéder au système : Engineering Mode & ADB **6**
3.1 Activer l'Engineering Mode sur la MG4 . . . 6
3.2 Connecter via ADB (câble USB) . . . 6
3.3 ADB over WiFi : sans câble USB (via `nmap`) . . . 6
3.3.1 Prérequis . . . 6
3.3.2 Étape 1 : Trouver l'IP de la MG4 avec nmap . . . 7
3.3.3 Étape 2 : Activer ADB over WiFi (première fois, câble requis) . . . 7
3.3.4 Étape 3 : Utilisation ensuite (sans câble) . . . 7
3.4 Réseau virtuel dédié pour la MG4 . . . 7
3.4.1 Principe . . . 8
3.4.2 Étape 1 : Activer le hotspot sur le telephone . . . 8
3.4.3 Étape 2 : Connecter le PC au hotspot . . . 8
3.4.4 Étape 3 : Connecter la MG4 au même hotspot . . . 8
3.4.5 Étape 4 : Connexion ADB sur le réseau virtuel . . . 8
3.5 Commandes ADB utiles . . . 9
3.6 Installer via clé USB (sans PC) . . . 9

4 Structure du projet **10**

5 Architecture Java : Concepts clés **11**
5.1 Héritage (`extends`) . . . 11
5.2 Interface (contrat entre classes) . . . 11
5.3 Classe anonyme (BroadcastReceiver) . . . 12
5.4 RecyclerView + Adapter + ViewHolder . . . 12
5.5 Persistance avec SharedPreferences . . . 13
5.6 AudioManager : Contrôle du son . . . 13

6 Build & Installation **14**
6.1 Ouvrir le projet dans Android Studio . . . 14
6.2 Builder l'APK . . . 14
6.3 Installer sur la MG4 . . . 14
6.4 Définir comme launcher par défaut . . . 14

7 Fonctionnalités **15**
7.1 Écran d'accueil (`HomeActivity`) . . . 15
7.2 Tiroir d'applications (`AppDrawerActivity`) . . . 15
7.3 Dialog long-press (`AppOptionsDialog`) . . . 15
7.4 Démarrage automatique (`BootReceiver`) . . . 15

8 Dépannage : Alarme de vitesse **16**
8.1 Étape 1 : Identifier le bon stream via ADB 16
8.2 Étape 2 : Mettre à jour le code 16
8.3 Étape 3 : Si l'alarme est gérée par SAIC (hors Android) 16
8.4 Étape 4 : Solution de secours : mode Ne pas déranger 16

9 Compatibilité & versions **18**
9.1 Palette de couleurs du launcher 18

▼ Avertissement important

Périmètre des modifications effectuées

Ce projet porte **exclusivement** sur le système d'infodivertissement (*infotainment*) de la MG4. Il s'agit de l'écran central Android du tableau de bord, utilisé pour la navigation, la musique et les réglages de confort.

- ✓ **Seul le système infotainment (Android 9, écran central) a été modifié.**
- ✓ L'installation se limite à un APK launcher sur la couche Android : aucun accès au firmware véhicule, au bus CAN, ni aux calculateurs embarqués.
- ✗ **Aucune modification n'a été apportée aux systèmes impactant la conduite** (direction, freinage, moteur, ADAS, ESP, boîte de vitesses, etc.).
- ✗ **Aucune modification n'a été effectuée sur les commandes au volant** (boutons, palettes, capteurs, régulateur de vitesse adaptatif, etc.).
- ✗ **Aucun comportement dangereux au volant n'a été adopté** lors des phases de test ou d'installation. Toutes les manipulations ont été effectuées véhicule à l'arrêt, moteur éteint ou en stationnement sécurisé.

Les modifications décrites dans ce document sont de nature **cosmétique et logicielle**, limitées à la personnalisation de l'interface utilisateur de l'écran central. Elles sont **entièrement réversibles** par simple désinstallation de l'APK et ne sauraient en aucun cas affecter la sécurité du véhicule, ses systèmes de sécurité actifs ou passifs, ni son homologation.
Cependant, la **garantie** de la voiture devient caduc.

1 Comprendre le système de la MG4

1.1 Analyse de l'OS

La MG4 embarque un système **Android 9 (API 28)** développé par **SAIC** (constructeur chinois de la MG). Ce n'est pas un Android "pur" (AOSP) — c'est une surcouche propriétaire avec :

- Des **applications système SAIC** pré-installées (navigation, musique, climatisation, etc.)
- Un **launcher natif SAIC** qui remplace l'écran d'accueil Android standard
- Des **services spécifiques** pour interfacer Android avec les capteurs de la voiture (vitesse, GPS, CAN bus)

1.2 Pourquoi peut-on installer un APK dessus ?

Parce que c'est Android. Même sur une voiture, Android permet d'installer des applications tierces via le débogage USB (ADB) ou une clé USB si les sources inconnues sont activées. La MG4 **n'a pas de Play Store** par défaut, donc toute installation se fait manuellement.

1.3 Ce qu'on ne peut PAS faire sans root

Action	Sans root	Avec root
Installer un APK	Oui (ADB/USB)	Oui
Remplacer le launcher	Oui (Intent HOME)	Oui
Désinstaller une app système	Non	Oui
Modifier des fichiers système	Non	Oui
Couper un service SAIC natif	Partiel (ADB)	Oui
Accès root shell	Non	Oui

▼ Attention

Rooter la MG4 passe par le déverrouillage du bootloader (méthode spécifique au firmware SAIC). Voir les forums MG4owners.co.uk et r/mg4. Ce document ne couvre pas cette étape. Cette étape est illégale et peut (et, en principe, va) nuire à la sécurité de l'usager.

2 Prérequis développement

2.1 Logiciels & Interfaces nécessaires

Voici ce que j'ai utilisé de la connexion à la voiture au build de l'APK du launcher :

Outil	Version recommandée	Rôle
Android Studio	Hedgehog (2023.1+)	IDE principal
Java JDK	**21**	Compilation
Gradle	**8.9**	Build system
Android Gradle Plugin (AGP)	**8.5.0**	Plugin Android
ADB	inclus avec Android Studio	Communication USB/WiFi

2.2 Compatibilité Java / Gradle

La voiture étant sous Android 9 (Android Automotive OS), il faut faire attention à la version de Java et la version de Gradle.

J'ai choisi Java 21 et donc Gradle 8.5.

Listing 1 – build.gradle (racine) + app/build.gradle

```
// build.gradle (racine)
classpath 'com.android.tools.build:gradle:8.5.0'

// app/build.gradle
compileSdk 35    // requis par AGP 8.5
minSdk     26
targetSdk  28    // version exacte de la MG4
```

▷ Conseil pratique

Dans Android Studio :
`Tools` → `SDK Manager` → `SDK Platforms` → cocher **Android 9.0 (API 28)** → Apply.

3 Accéder au système : Engineering Mode & ADB

3.1 Activer l'Engineering Mode sur la MG4

La première étape est d'activer l'équivalent du "Mode développeur" de la voiture.

1. Ouvrir **Paramètres** sur l'écran de la MG4
2. Aller dans les informations systèmes
3. Trouver **"Numéro de build Infotainment"** ou quelque-chose du genre
4. **Taper** X **fois dessus** (entre 4 et 7) → un mot de passe sera demandé

Ce mot de passe varie selon les modèles exacts des voitures.

Le mot de passe de l'Engineering Mode a été obtenu par analyse des logs système exportés via ADB, et ensuite divulgué sur des forums (là où nous l'avons trouvé).

> Une fois dans le Engineering Mode, il faut activer l'ADB (ou le Débogage USB suivant la version de la voiture).

3.2 Connecter via ADB (câble USB)

On peut à priori connecter directement certaines voitures et l'ordinateur via le Port Data de la voiture (= le port sous l'écran).

Listing 2 – Connexion ADB USB

```
# Brancher la MG4 au PC via USB (port USB-A du tableau de bord)
adb devices

# Reponse attendue :
# List of devices attached
# XXXXXXXX     device     <- ID de la voiture
```

> Si l'état est `unauthorized` → une popup s'affiche sur l'écran de la MG4. Accepter.
>
> S'il y a une ligne vide, il peut y avoir énormément de raisons et de fixs.
> Si rien ne marche, la voiture ne reconnaît simplement pas cette connexion et il faut essayer un autre moyen.

3.3 ADB over WiFi : sans câble USB (via `nmap`)

Plutôt que de brancher un câble USB à chaque fois, on peut faire du **ADB over WiFi** en se connectant directement à l'IP de la MG4 sur le port **5555**.

3.3.1 Prérequis

— Le PC et la MG4 doivent être sur le **même réseau WiFi**

— L'ADB / débogage USB doit avoir été activé **au moins une fois** via câble (pour autoriser le PC)

3.3.2 Étape 1 : Trouver l'IP de la MG4 avec nmap

Si l'IP de la MG4 n'est pas connue, scanner le réseau local :

Listing 3 – Scan réseau avec nmap

```
nmap -sn 192.168.1.0/24

# Exemple de sortie :
# Nmap scan report for 192.168.1.42
# Host is up (0.023s latency).
# MAC Address: XX:XX:XX:XX:XX:XX (SAIC Motor)
```

Vérifier ensuite que le port 5555 (ADB) est ouvert sur l'IP trouvée :

Listing 4 – Vérification port ADB

```
nmap -p 5555 192.168.1.42

# PORT      STATE   SERVICE
# 5555/tcp  open    freeciv    <- ADB est accessible
```

3.3.3 Étape 2 : Activer ADB over WiFi (première fois, câble requis)

Listing 5 – Activation ADB WiFi

```
# Brancher en USB une premiere fois
adb tcpip 5555

# Debrancher le cable, puis se connecter en WiFi
adb connect 192.168.1.42:5555
```

3.3.4 Étape 3 : Utilisation ensuite (sans câble)

À chaque session, si déconnecté :

```
adb connect 192.168.1.42:5555
adb install -r app-debug.apk
adb logcat
```

▼ Attention

L'IP de la MG4 peut changer si le bail DHCP expire. Si `adb connect` échoue, relancer le scan nmap pour retrouver la nouvelle IP.

3.4 Réseau virtuel dédié pour la MG4

Pour isoler la connexion ADB over WiFi et garantir une IP stable (évitant les conflits avec d'autres appareils sur le réseau), j'ai créé un réseau virtuel.

3.4.1 Principe

Au lieu de connecter le PC et la MG4 sur votre box internet (ou l'IP change selon le DHCP), on créé un **réseau privé intermédiaire** :

`PC` ⟷ `Hotspot téléphone` ⟷ `MG4`

Avantages :

- IP stables et prédictibles (plage `192.168.43.x` sur Android)
- Réseau isolé → aucun conflit avec d'autres appareils
- Utilisable en parking

3.4.2 Étape 1 : Activer le hotspot sur le telephone

1. **Paramètres** → **Point d'accès et partage de connexion** → **Activer le hotspot WiFi**
2. Configurer un SSID et mot de passe facilement mémorisables
3. Privilégier le **canal 2.4 GHz** pour une meilleure portée (utile en parking souterrain)

3.4.3 Étape 2 : Connecter le PC au hotspot

Listing 6 – Vérification de l'interface réseau du PC

```
# Windows
ipconfig

# Linux / macOS
ip addr show

# L'IP du PC sera du type 192.168.43.x (plage typique des hotspots
    Android)
```

3.4.4 Étape 3 : Connecter la MG4 au même hotspot

1. Sur l'écran MG4 : **Paramétrés** → **WiFi** → rejoindre le hotspot du téléphone
2. L'IP attribuée à la MG4 est visible dans les paramètres WiFi de la MG4, ou via `nmap`

Listing 7 – Scanner le réseau hotspot

```
nmap -sn 192.168.43.0/24

# Nmap scan report for 192.168.43.87
# Host is up (0.031s latency).
# MAC Address: XX:XX:XX:XX:XX:XX (SAIC Motor)
```

3.4.5 Étape 4 : Connexion ADB sur le réseau virtuel

Listing 8 – Connexion ADB via le réseau hotspot

```
adb connect 192.168.43.87:5555
adb devices

# List of devices attached
# 192.168.43.87:5555     device
```

▼ Attention

Les paquets échanges entre le PC et la MG4 restent **en local** sur le réseau hotspot et ne transitent pas par la data mobile. Vérifier toutefois selon votre opérateur.

3.5 Commandes ADB utiles

Listing 9 – Référence commandes ADB

```
# Installer un APK
adb install -r app/build/outputs/apk/debug/app-debug.apk

# Lister toutes les apps installees
adb shell pm list packages

# Apps systeme SAIC
adb shell pm list packages | grep -i saic
adb shell pm list packages | grep -i mg

# Logs en temps reel
adb logcat -s MG4Launcher

# Etat audio
adb shell dumpsys audio | grep -E 'stream|volume|mute'

# Couper un stream audio
adb shell media volume --stream 1 --set 0    # STREAM_SYSTEM
adb shell media volume --stream 4 --set 0    # STREAM_ALARM

# Forcer l'arret d'un service
adb shell am force-stop com.saic.NOM_DU_PACKAGE

# Transfert de fichiers
adb pull /sdcard/fichier.txt ./fichier.txt
adb push monfichier.apk /sdcard/
```

3.6 Installer via clé USB (sans PC)

Copier l'APK sur une clé USB FAT32, la brancher sur la MG4, et utiliser le gestionnaire de fichiers intégré pour lancer l'installation.

▷ Conseil pratique

Il faut activer l'installation d'applications de Sources inconnues pour pouvoir installer des APKs.
Faire très attention à ce qu'on installe car c'est du développement critique.

4 Structure du projet

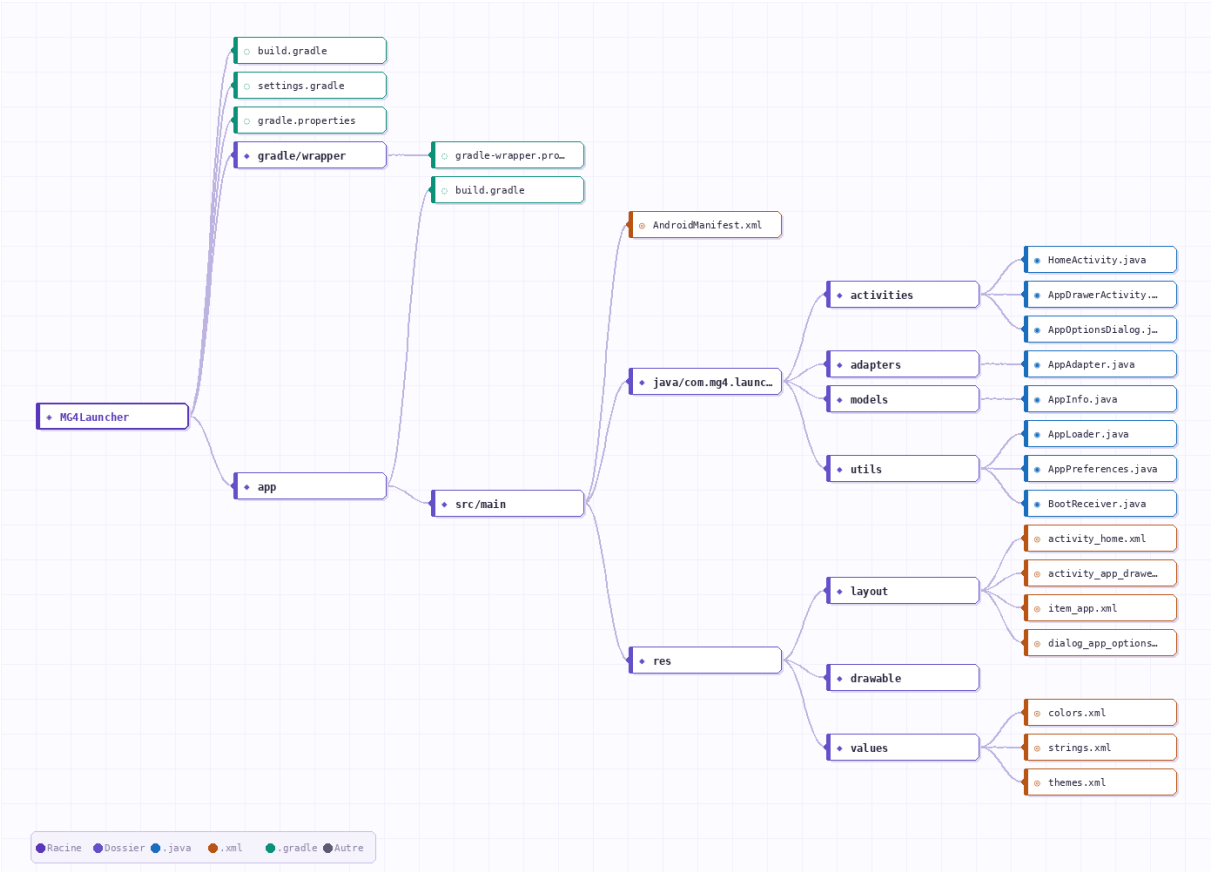


FIGURE 1 – Structure du projet MG4Launcher

5 Architecture Java : Concepts clés

5.1 Héritage (extends)

Toutes les activités héritent de `AppCompatActivity` :

Listing 10 – Héritage et cycle de vie Android

```
// HomeActivity herite de AppCompatActivity
// qui herite de Activity -> Context -> ...
public class HomeActivity extends AppCompatActivity {

    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState); // appel obligatoire au
   parent
        setContentView(R.layout.activity_home);
    }
}
```

Σ Éclairage technique

Le **cycle de vie Android** est géré par la classe parente :

- `onCreate()` → `onStart()` → `onResume()` : app visible et active
- `onPause()` → `onStop()` → `onDestroy()` : app qui disparaît

Sur un launcher, on ne doit jamais vraiment quitter (`onDestroy`), d'où le `launchMode="singleTask"` dans le Manifest.

5.2 Interface (contrat entre classes)

`AppOptionsDialog` définit une interface `Listener` que l'activité appelante doit implémenter. C'est un **contrat** : "tu veux le dialog ? Tu dois fournir ces 4 méthodes."

Listing 11 – Interface Listener dans AppOptionsDialog

```
// Definition de l'interface
public interface Listener {
    void onPin(AppInfo app);
    void onUnpin(AppInfo app);
    void onFavorite(AppInfo app);
    void onUninstall(AppInfo app);
}

// HomeActivity "remplit le contrat" via une classe anonyme
AppOptionsDialog dialog = new AppOptionsDialog(this, app, true,
    new AppOptionsDialog.Listener() {
        @Override
        public void onPin(AppInfo a) {
            prefs.addPinned(a.getPackageName());
            reloadApps();
        }
        // ... les 3 autres methodes obligatoires
    });
```

5.3 Classe anonyme (BroadcastReceiver)

Un `BroadcastReceiver` écoute des événements système. On le déclare **en ligne** sans créer un fichier séparé :

Listing 12 – BroadcastReceiver anonyme

```
private final BroadcastReceiver appChangeReceiver = new
    BroadcastReceiver() {
     @Override
     public void onReceive(Context context, Intent intent) {
         reloadApps(); // rafraichir quand une app est installee/
    supprimee
     }
};

// Enregistrer dans onResume()
registerReceiver(appChangeReceiver, filter);

// Desenregistrer dans onPause() pour eviter les fuites memoire
unregisterReceiver(appChangeReceiver);
```

5.4 RecyclerView + Adapter + ViewHolder

C'est le pattern standard Android pour afficher des listes performantes :

> `RecyclerView` (la grille) → `AppAdapter` (gère les données) → `ViewHolder` (représente UNE cellule)

Listing 13 – Pattern RecyclerView

```
public class AppAdapter extends RecyclerView.Adapter<AppAdapter.
    ViewHolder> {

     // ViewHolder = classe interne statique qui "tient" les vues d'un
    item
     public static class ViewHolder extends RecyclerView.ViewHolder {
         ImageView ivIcon;
         TextView tvName;

         public ViewHolder(View itemView) {
             super(itemView);
             ivIcon = itemView.findViewById(R.id.iv_app_icon);
             tvName = itemView.findViewById(R.id.tv_app_name);
         }
     }

     @Override
     public void onBindViewHolder(ViewHolder holder, int position) {
         AppInfo app = appList.get(position);
         holder.tvName.setText(app.getAppName());
         holder.ivIcon.setImageDrawable(app.getIcon());
     }
}
```

5.5 Persistance avec SharedPreferences

`SharedPreferences` = stockage clé/valeur sur le disque, survivant aux redémarrages.

Listing 14 – SharedPreferences — lecture et écriture

```
// Ecriture
prefs.edit()
      .putStringSet("pinned_apps", setDePackageNames)
      .apply(); // asynchrone (recommande) vs .commit() synchrone

// Lecture
Set<String> pinnedPackages = prefs.getStringSet("pinned_apps", new
   HashSet<>());
```

⋆ Exemple concret

Sans `SharedPreferences`, `AppLoader.loadApps()` recréait des objets `AppInfo` neufs à chaque appel → `setPinned(true)` en mémoire était perdu au prochain rechargement. Donc, épingler une app à l'écran d'accueil du launcher ne fonctionnait pas.

5.6 AudioManager : Contrôle du son

On peut à priori désactiver certains signaux même sans Root :

Listing 15 – Streams audio Android

```
AudioManager am = (AudioManager) getSystemService(Context.
   AUDIO_SERVICE);

// Streams disponibles
AudioManager.STREAM_SYSTEM        // 1 - sons systeme, bips
AudioManager.STREAM_RING          // 2 - sonnerie
AudioManager.STREAM_NOTIFICATION  // 3 - notifications
AudioManager.STREAM_ALARM         // 4 - alarmes Android
AudioManager.STREAM_MUSIC         // 5 - musique / media

// Couper un stream
int original = am.getStreamVolume(AudioManager.STREAM_SYSTEM);
am.setStreamVolume(AudioManager.STREAM_SYSTEM, 0, 0);

// Restaurer
am.setStreamVolume(AudioManager.STREAM_SYSTEM, original, 0);
```

6 Build & Installation

Je trace ici le build (très classique) du projet en APK :

6.1 Ouvrir le projet dans Android Studio

1. `File` → `Open` → sélectionner le dossier `MG4Launcher/`
2. Attendre la sync Gradle
3. Si erreur de sync → vérifier les versions Java/Gradle (section 2)

6.2 Builder l'APK

`Build` → `Build Bundle(s) / APK(s)` → `Build APK(s)`

L'APK généré se trouve dans :

```
app/build/outputs/apk/debug/app-debug.apk
```

6.3 Installer sur la MG4

On peut installer l'APK directement via Clé USB FAT32, ou via ADB :

Listing 16 – Installation via ADB

```
# Via ADB (recommande)
adb install -r app/build/outputs/apk/debug/app-debug.apk

# Verifier que l'installation a reussi
adb shell pm list packages | grep mg4
```

6.4 Définir comme launcher par défaut

Au prochain appui sur le bouton **Home** de la MG4, Android affichera : *"Choisir une application pour l'action Home"* → sélectionner **MG4 Launcher** → **Toujours**.

7 Fonctionnalités

7.1 Écran d'accueil (`HomeActivity`)

- Horloge temps réel (`TextClock`)
- Grille 4 colonnes d'apps épinglées
- Bouton "Toutes les apps" → ouvre le Drawer d'applications
- Bouton **cloche** / **cloche barrée** → coupe/réactive le son système (alarme vitesse)
- La barre de navigation MG4 reste visible (pas de plein écran)

7.2 Tiroir d'applications (`AppDrawerActivity`)

- Grille 5 colonnes de toutes les apps installées
- Recherche temps réel (filtrage au fur et à mesure de la frappe)

7.3 Dialog long-press (`AppOptionsDialog`)

Maintenir appuyé sur une app ouvre un menu avec :

Option	Comportement
Épingler sur l'accueil	Ajoute l'app à la grille de la home
Retirer de l'accueil	Retire l'app de la grille
Ajouter aux favoris	Marque l'app comme favorite (persisté)
Désinstaller	Ouvre la popup système Android
Annuler	Ferme le dialog

▷ Conseil pratique

Les états **épinglé** et **favori** sont persistants grâce `SharedPreferences`.

7.4 Démarrage automatique (`BootReceiver`)

Le launcher se relance automatiquement au démarrage de la voiture via `BOOT_COMPLETED` :

Listing 17 – AndroidManifest.xml - BootReceiver

```
<uses-permission android:name="android.permission.
   RECEIVE_BOOT_COMPLETED"/>

<receiver android:name=".utils.BootReceiver" android:exported="true">
    <intent-filter>
        <action android:name="android.intent.action.BOOT_COMPLETED"/>
    </intent-filter>
</receiver>
```

8 Dépannage : Alarme de vitesse

Le bouton cloche coupe `STREAM_SYSTEM` et `STREAM_NOTIFICATION`.
Si l'alarme continue de sonner, suivre les étapes ci-dessous.

8.1 Étape 1 : Identifier le bon stream via ADB

Listing 18 – Identification du stream audio

```
# Observer les streams au moment ou l'alarme sonne
adb shell dumpsys audio | grep -E 'stream|volume'

# Tester chaque stream un par un (0 = muet)
adb shell media volume --stream 1 --set 0   # SYSTEM
adb shell media volume --stream 2 --set 0   # RING
adb shell media volume --stream 3 --set 0   # NOTIFICATION
adb shell media volume --stream 4 --set 0   # ALARM
adb shell media volume --stream 5 --set 0   # MUSIC

# Restaurer apres test
adb shell media volume --stream X --set 15
```

8.2 Étape 2 : Mettre à jour le code

Une fois le bon stream identifié, modifier `toggleSpeedAlarm()` dans `HomeActivity.java` :

```
audioManager.setStreamVolume(AudioManager.STREAM_XXXX, 0, 0); //
    remplacer XXXX
```

8.3 Étape 3 : Si l'alarme est gérée par SAIC (hors Android)

Listing 19 – Identification service SAIC

```
adb shell dumpsys activity services | grep -i speed
adb shell dumpsys activity services | grep -i alarm
adb shell pm list packages | grep -i saic

# Forcer l'arret
adb shell am force-stop com.saic.NOM_DU_PACKAGE
```

8.4 Étape 4 : Solution de secours : mode Ne pas déranger

Listing 20 – AndroidManifest.xml — Permission DND

```
<uses-permission android:name="android.permission.
    ACCESS_NOTIFICATION_POLICY"/>
```

Listing 21 – Mode "Ne pas déranger" via NotificationManager

```
NotificationManager nm =
    (NotificationManager) getSystemService(NOTIFICATION_SERVICE);

nm.setInterruptionFilter(
    NotificationManager.INTERRUPTION_FILTER_NONE); // DND ON

nm.setInterruptionFilter(
    NotificationManager.INTERRUPTION_FILTER_ALL);  // DND OFF
```

▷ Conseil pratique

Sur Android 9 (API 28), `ACCESS_NOTIFICATION_POLICY` n'est pas considérée comme dangereuse : elle ne nécessite pas d'approbation runtime, juste la déclaration dans le Manifest.

9 Compatibilité & versions

Composant	Version
Android cible (MG4)	**9.0 (API 28)**
Android minimum supporté	8.0 (API 26)
Java	21
Gradle	8.9
AGP	8.5.0
compileSdk	35
targetSdk	28

9.1 Palette de couleurs du launcher

Nom	Hex	Usage
`accent_green`	`#00E676`	Boutons principaux, options dialog
`bg_dark`	`#0D0D0D`	Fond principal
`surface`	`#1A1A1A`	Cartes, éléments secondaires
`text_secondary`	`#9E9E9E`	Labels, textes discrets